COMP 2011 Midterm Exam - Spring 2025 - HKUST

Date: March 22, 2025 (Saturday)
Time Allowed: 2 hours, 11:00am – 1:00pm
Instructions:

1. This is a closed-book, closed-notes examination.
2. There are **8** questions on **22** pages (including this cover page, honor code and 3 blank pages at the end).
3. Write your answers in the space provided in black/blue ink. *NO pencil please, otherwise you are not allowed to appeal for any grading disagreements.*
4. All programming codes in your answers must be written in the ANSI C++11 version as taught in the class.
5. For programming questions, you are **NOT** allowed to define additional helper functions or structures, nor global variables, unless otherwise stated. You **cannot** use any library functions not mentioned, nor the `auto` keyword in defining identifiers in the questions.
6. Marks will be deducted if your codes produce memory leaks.

Student Name	
Student ID	
Seat Number	

Problem	**Score**
1. Const-ness	/ 10
2. Break and Continue	/ 8
3. Switch-Case Statement	/ 6
4. Parameter Passing	/ 10
5. Recursion	/ 8
6. Control Flow	/ 8
7. Array	/ 25
8. C String	/ 25
Total	/ 100

The HKUST Academic Honor Code

Honesty and integrity are central to the academic work of HKUST. Students of the University must observe and uphold the highest standards of academic integrity and honesty in all the work they do throughout their program of study.

As members of the University community, students have the responsibility to help maintain the academic reputation of HKUST in its academic endeavors.

Sanctions will be imposed if students are found to have violated the regulations governing academic integrity and honesty.

Declaration of Academic Integrity

I confirm that I have answered the questions using only materials specifically approved for use in this examination, that all the answers are my own work, and that I have not received any assistance during the examination.

Signature: _____

Problem 1 [10 points] Const-ness

```
1  #include <iostream>
2  using namespace std;
3
4  void f1(int& a) { cout << ++a << endl; }
5
6  void f2(const int& a) { cout << a << endl; }
7
8  int f3() { int a = 99; return a; }
9
10 int main() {
11    const int A = 100, B = 100;
12    int w = 12, x = 12, y = 12, z = 12;
13
14    /* This comment is to be replaced by a function call statement in the question */
15    return 0;
16 }
```

If line #14 of the above program is replaced by each of the following function calls, write down the corresponding output if the resulting program can be compiled successfully. If the resulting program cannot be compiled successfully, write "compilation error". If it compiles but may crash when it is run, write "runtime error".

	Function call	**Output or "compilation error" or "runtime error"**
(a)	`f1( 10 );`	
(b)	`f2( 10 );`	
(c)	`f1( A );`	
(d)	`f2( B );`	
(e)	`f1( w );`	
(f)	`f2( x );`	
(g)	`f1( y * 2 );`	
(h)	`f2( z * 2 );`	
(i)	`f1( f3() );`	
(j)	`f2( f3() );`	

Problem 2 [8 points] Break and Continue

What is the output of the following program?

```
#include <iostream>
using namespace std;

int main() {
    char arr[4][4] = {
        {'H', 'E', 'L', 'L'},
        {'O', ' ', 'W', 'O'},
        {'R', '!', 'L', 'D'},
        {'!', ' ', ' ', ' '}
    };

    char word[10] = {'$','$','$','$','$','$','$','$','$','$'};
    int index = 0;

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (arr[i][j] == ' ') {
                continue;
            }
            if (arr[i][j] == '!') {
                break;
            }
            word[index]= arr[i][j];
            index++;
        }
    }
    word[index] = '\0';
    cout << word << endl;
    return 0;
}
```

Answer: _____

Problem 3 [6 points] Switch-Case Statement

What is the output of the following program?

If the output consists multiple lines, you need to put them in different lines.

```
#include <iostream>
using namespace std;

enum Operation { ADD, SUBTRACT, MULTIPLY, DIVIDE };

int main() {
    int result = 0;
    for (int i = 1; i <= 4; i++) {
        Operation op = static_cast<Operation>(i%4);
        switch (op) {
            case ADD:
                for (int j = 1; j <= 3; j++) {
                    result += j;
                }
            case SUBTRACT:
                result -= 2;
                break;
            case MULTIPLY:
                result *= 2;
                break;
            case DIVIDE:
                result /= 2;
                break;
        }
    cout << result << endl;
    }
    return 0;
}
```

Answer: _____

Problem 4 [10 points] Parameter Passing

What are the outputs of the following programs?

(a)
```
#include <iostream>
using namespace std;

void processValues(int x, int &y) {
    x += 5;
    y *= 2;
}

int main() {
    int a = 10;
    int b = 5;

    processValues(a, b);
    processValues(b, a);
    cout << a << '\t' << b << endl;

    return 0;
}
```

Answer: _____

(b)
```
#include <iostream>
using namespace std;

void processArray(int arr[], int size, int &sum) {
    for (int i = 0; i < size; i++)
        if (arr[i] % 2 == 0)
            sum += arr[i];
        else
            arr[i] *= 2;
}

int main() {
    int numbers[5] = {1, 2, 3, 4, 5};
    int sum = 0;

    processArray(numbers, 5, sum);
    processArray(numbers, 5, sum);
    cout << sum << endl;

    return 0;
}
```

Answer: _____

(c)
```cpp
#include <iostream>
using namespace std;

int& getValue(int& x) {
    switch (x) {
        case 1:
            x += 10;
            break;
        case 2:
            x += 20;
            break;
        default:
            x += 30;
            break;
    }
    return x;
}

int main() {
    int x = 2;
    getValue(x) += 5;
    cout << x << endl;
    return 0;
}
```

Answer: _____

Problem 5 [8 points] Recursion

Write a recursive function with the following header: `max_digit(const int a)`, which finds out the greatest digit in a given positive integer a. Below is a sample driver program that calls the recursive function.

```
#include <iostream>
using namespace std;

int max_digit(const int a)
{ // ANSWER: Your implementation MUST BE RECURSIVE.









}

int main()
{
    int x;              // Assume that only positive integers will be used
    while (cin >> x)
        cout << "Output: " << max_digit(x) << endl;
   return 0;
}
```

And here are some sample inputs and the expected outputs.

```
3456
Output: 6
6543
Output: 6
3645
Output: 6
4
Output: 4
333
Output: 3
286305067
Output: 8
```

Problem 6 [8 points] Control Flow

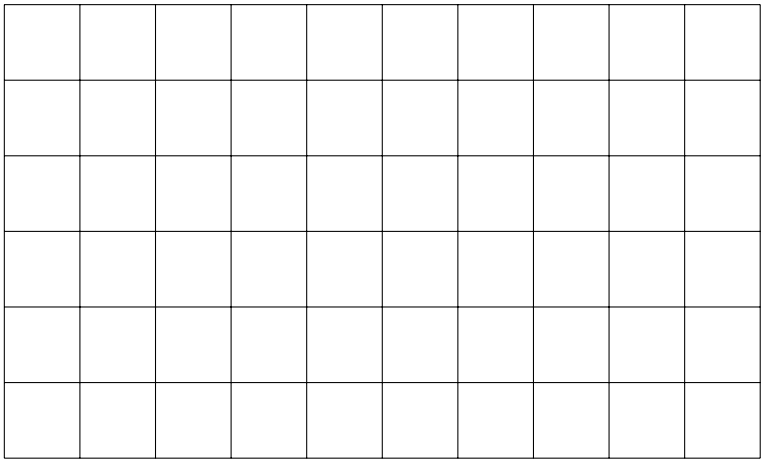
Draw the shape that is printed by the following function.

You can use the grid provided for better alignment. Note that the grid space is more than enough.

You only need to draw the visible characters. For an empty space, e.g. `cout << " ";`, you can just leave the corresponding grid cell empty. The printing should start from the leftmost part as if this is a terminal. You should not leave empty spaces on the left unless the code prints white spaces.

```
void printShape(int height) {   // height = 4
    for (int i = 0; i < height; i++) {
        for (int j = 0; j < height - i - 1; j++) {
            cout << " ";
        }
        if (i == 0) {
            cout << "*";
        } else if (i == height - 1) {
            for (int j = 0; j < 2 * i + 1; j++) {
                cout << "*";
            }
        } else {
            cout << "*";
            for (int j = 0; j < 2 * i - 1; j++) {
                cout << " ";
            }
            cout << "*";
        }
        cout << endl;
    }
}
```

Answer:



Problem 7 [25 points] Array

This question deals with 1D and 2D arrays, and the 2D arrays are also always square matrices. Although the arrays in the code below are character arrays, the ideas should work for arrays of different data types as well. Implement the following 3 functions:

(a) `void convert_1D_to_2D(const char v[], char m[][N], const int size)`
The function converts a given 1D array **v** to a 2D submatrix of dimension $size \times size$ in the given matrix **m** (of dimension $N \times N$).
Assumption: $size \geq 0$ and $size \leq N$; dimension of **v** is always $\geq size \times size$.

(b) `void print_submatrix(const char m[][N], const int size, const int start_row, const int start_col, const int subsize)`
Although the given square matrix **m** can hold a maximum of $N \times N$ items, only the first *size* of rows and columns are actually filled. The function prints the content in the "imaginary" square submatrix of **m**, which starts at `m[start_row][start_col]` (i.e., its upper-left corner item is `m[start_row][start_col]`) and has a dimension of $subsize \times subsize$.
Assumption: $size \geq 0$ and $size \leq N$; $start_row \geq 0$; $start_col \geq 0$.
Note: If the required submatrix goes beyond the given matrix of $size \times size$, only print the items within the filled matrix of dimension $size \times size$.

(c) `void clockwise_ring_rotate(char m[][N], const int size)`
The (outermost) ring of a matrix consists of the first row, first column, last row and last column. This function implements a clockwise rotation of the outermost ring of the given square matrix **m** of dimension $size \times size$ by one step. Matrix items inside the ring should remain unmodified.
Assumption: $size \geq 2$ and $size \leq N$.

Important Requirement: In the implementation of all functions in this question, you are **NOT** allowed to define/declare any local/global (1D, 2D, or any high-dimensional) arrays, and you can only use those given in the formal parameter list.

An example driver program is given below. Although the dimension N and some arguments to the 3 required functions are hardcoded in the given code (e.g., $N = 8$), your code should work for arbitrary values which are only limited by the given requirements and assumptions.

```
#include <iostream>
using namespace std;
const int N = 8;

/* Implement the following 3 functions */
void convert_1D_to_2D(const char v[], char m[][N], const int size) { /*(a) TODO*/ }
void print_submatrix(const char m[][N], const int size, const int start_row,
                     const int start_col, const int subsize) { /*(b) TODO*/ }
void ring_rotate_clockwise(char m[][N], const int size) { /*(c) TODO*/ }

int main()
{
    char v[N * N];      // 1D char array
    char m[N][N];       // 2D square char array
    const int L = 5;    // Assumption: 0 <= L <= N

    // Input L*L char to the 1D char array, v
    for (int k = 0; k < L*L; ++k)
        cin >> v[k];

    convert_1D_to_2D(v, m, L); // Note: only the first L rows/columns are used
    cout << "\n## L*L char matrix, m, converted from 1D char array, v ##" << endl;
    print_submatrix(m, L, 0, 0, L);

    cout << "\n## 3x3 sub-matrix of m starting at m[1][1] ##" << endl;
    print_submatrix(m, L, 1, 1, 3);

    cout << "\n## 5x5 sub-matrix of m starting at m[3][4] ##" << endl;
    print_submatrix(m, L, 3, 4, 5);

    convert_1D_to_2D(v, m, L);   // Re-store the square matrix, m, from array, v
    clockwise_ring_rotate(m, L); // Rotate the outermost ring of L*L matrix, m
    cout << "\n## matrix m after its clockwise ring rotation ##" << endl;
    print_submatrix(m, L, 0, 0, L);

    clockwise_ring_rotate(m, 3); // Rotate the outermost ring of m's 3x3 sub-matrix
    cout << "\n## matrix m after clockwise ring rotation of its 3*3 submatrix ##"
         << endl;
    print_submatrix(m, L, 0, 0, L);
    return 0;
}
```

Given the input "ABCDEFGHIJKLMNOPQRSTUVWXYZ", the program prints the following.

```
## L*L char matrix, m, converted from 1D char array, v ##
ABCDE
FGHIJ
KLMNO
PQRST
UVWXY

## 3x3 sub-matrix of m starting at m[1][1] ##
GHI
LMN
QRS

## 5x5 sub-matrix of m starting at m[3][4] ##
T
Y

## matrix m after its clockwise ring rotation ##
FABCD
KGHIE
PLMNJ
UQRSO
VWXYT

## matrix m after clockwise ring rotation of its 3*3 submatrix ##
KFACD
PGBIE
LMHNJ
UQRSO
VWXYT
```

Answer:

(a) [4 points] `void convert_1D_to_2D(const char v[], char m[][N], const int size)`

(b) [8 points] `void print_submatrix(const char m[][N], const int size, const int start_row, const int start_col, const int subsize)`

(c) [13 points] For this part, you **MUST** write **ONE** comment line for the modification of each of the first/last row/column (i.e., totally 4 lines of comments) to show how you perform the ring rotation; marks will be deducted if you do not.

```
void clockwise_ring_rotate(char m[][N], const int size)
```

Problem 8 [25 points] C String

In this question, you are asked to complete the implementation of a program to extract words from some sentences. Each sentence is stored as a C string. Each sentence will only consist of alphabets (A to Z, a to z), single spaces, and some punctuation (including comma (,), full stop (.), exclamation mark (!), and question mark (?)). There may be consecutive combinations of spaces and punctuation.

You will need to define the functions, namely, `is_alphabet()`, `convert_to_lowercase()` and `extract_nth_word()`, which are called in the `main()` function:

```
#include <iostream>
using namespace std;

const int MAX_LEN = 100;
const int MAX_WORDS = 20;

// Definition of other functions go here ...
#include "functions.cpp"

int main() {
   cout << boolalpha; // To print booleans in English
   cout << "Is N an alphabet? " << is_alphabet('N') << endl;
   cout << "Is n an alphabet? " << is_alphabet('n') << endl;
   cout << "Is single space an alphabet? " << is_alphabet(' ') << endl;
   cout << "Is exclamation mark an alphabet? " << is_alphabet('!') << endl;
   cout << endl;

   char sentences[3][MAX_LEN] = {"   To be, or NOT to be?",
                                 "Happiness is not by chance,BUT by choice!  ",
                                 "It always seems impossible until     it is done..."
                                };
   char result[MAX_LEN];
   cout << "Printing the sentences in lowercase characters:" << endl;
   for (int i=0; i<3; i++) {
      convert_to_lowercase(sentences[i], result);
      cout << result << endl;
   }
   cout << endl;

   char word[MAX_LEN];
   char word_in_lowercase[MAX_LEN];
   for (int i=0; i<3; i++) {
      cout << "Extracting sentence " << i << ": " << sentences[i] << endl;
      for (int j=0; j<MAX_WORDS; j++)
         if ( extract_nth_word(sentences[i], j, word) ) {
            convert_to_lowercase(word, word_in_lowercase);
            cout << j << "-th word: " << word_in_lowercase << endl;
```

```
38          }
39      }
40      return 0;
41  }
```

Here is the output of the program:

```
Is N an alphabet? true
Is n an alphabet? true
Is single space an alphabet? false
Is exclamation mark an alphabet? false

Printing the sentences in lowercase characters:
   to be, or not to be?
happiness is not by chance,but by choice!
it always seems impossible until     it is done...

Extracting sentence 0:    To be, or NOT to be?
0-th word: to
1-th word: be
2-th word: or
3-th word: not
4-th word: to
5-th word: be
Extracting sentence 1: Happiness is not by chance,BUT by choice!
0-th word: happiness
1-th word: is
2-th word: not
3-th word: by
4-th word: chance
5-th word: but
6-th word: by
7-th word: choice
Extracting sentence 2: It always seems impossible until     it is done...
0-th word: it
1-th word: always
2-th word: seems
3-th word: impossible
4-th word: until
5-th word: it
6-th word: is
7-th word: done
```

Note:

- Your implementation should give the correct output as shown above.
- Your implementation should also be able to give correct results even if the values of the global constants, `MAX_LEN` and `MAX_WORDS`, are changed.
- Your implementation should also be able to handle different contents in the `sentences` array.
- You are NOT allowed to use any of the C string library functions.
- Whenever a function argument should not be modified, you are required to specify it with `const`, otherwise marks will be deducted.

Answer:

(a) [4 points] Implement the function `bool is_alphabet(const char c)` to return `true` if the given character, `c`, is an alphabet; otherwise, return `false`.

```
bool is_alphabet(const char c)
{
  // Add your code here









}
```

(b) [8 points] Define the `convert_to_lowercase()` function. The function converts all alphabets from uppercase to lowercase. You may use the function in part (a) if necessary.

```
/* Complete the function header and implement convert_to_lowercase() here */

void convert_to_lowercase(                                                    )
{









}
```

(c) [13 points] Define the `extract_nth_word()` function. The function will extract the n-th word (counted from zero) from a given sentence. The function will return `true` if the n-th word can be extracted successfully. It will return `false` if the n-th word does not exist. The function will ignore any leading and trailing single spaces and punctuation. You may use the functions in parts (a) and (b) if necessary.
For this part, you **MUST** write enough comments to describe your code, otherwise marks will be deducted.

```
/* Complete the function header and implement extract_nth_word() here */

            extract_nth_word(                                                    )
{









}
```

---------- END OF PAPER ----------

/* Rough work */

/* Rough work */